



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2133 — Estructuras de Datos y Algoritmos 1'2026

Interrogación 2

17 de junio de 2026

Tiempo: 2 horas

Condiciones de entrega: Se deben entregar 3 de las 4 preguntas.

Forma de Entrega: La entrega es a través de Canvas. Al final de la prueba tienen 10 minutos para subir la imagen de la prueba. En formato vertical, cada pregunta y el torpedo por separado.

Dispositivo: Para digitalizar la prueba se permite el uso de UN dispositivo (teléfono, tablet) el cual debe estar guardado en el bolso y en silencio durante la prueba. En caso de ir al baño el dispositivo debe dejarse adelante en la mesa con los ayudantes.

Torpedo: Se permite una hoja tamaño oficio escrita a mano por ambas caras.

Evaluación: Cada pregunta tiene 6 puntos (+1 punto base). La nota es el promedio de las 3 preguntas entregadas. La nota de la interrogación es el promedio de las 3 preguntas entregadas.

Hint: Lea todas las preguntas antes de iniciar las respuestas.

1. Grafos

- I. (3.0 pts) Dado el grafo dirigido $G = (V, E)$ con

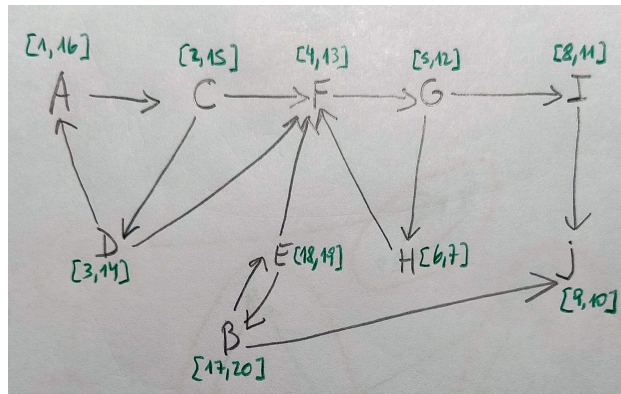
$$V = \{A, B, C, D, E, F, G, H, I, J\}$$

$$E = \{(A, C), (C, D), (D, A), (B, E), (E, B), \\ (F, G), (G, H), (H, F), (C, F), (D, F), \\ (E, F), (B, J), (G, I), (I, J)\},$$

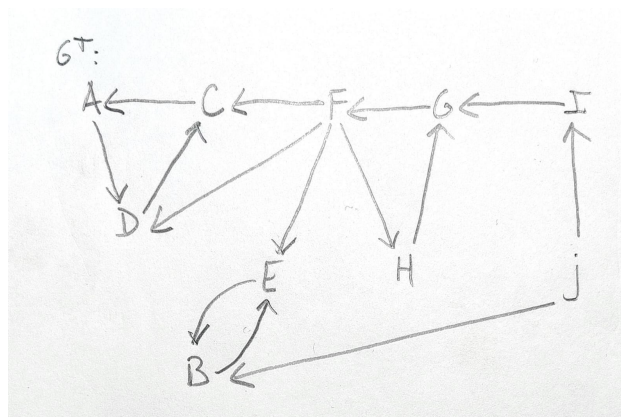
ejecute el algoritmo de Kosaraju para calcular sus componentes fuertemente conectadas. Para mostrar la ejecución del algoritmo, dibuje la siguiente información: (i) intervalos DFS de los nodos, (ii) grafo transpuesto, (iii) orden en que se procesan los nodos, y (iv) los representantes (arbitrarios) de cada componente.

Solución:

(i): Como el enunciado no fija un orden único para el DFS, se aceptan distintos conjuntos de intervalos, siempre que sean coherentes con una ejecución válida de DFS y que el orden usado en Kosaraju sea consistente con esos intervalos. Una posible respuesta, comenzando el DFS desde el nodo A , es la siguiente:



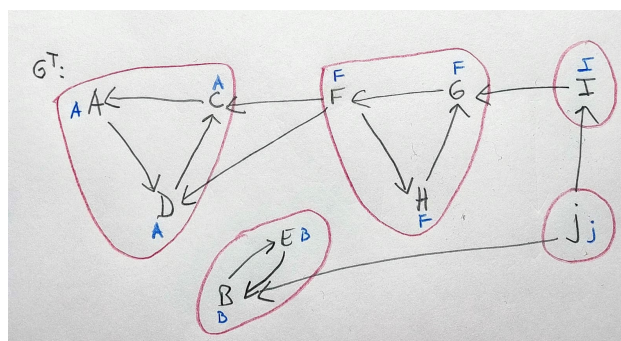
(ii): El grafo transpuesto G^T es el siguiente:



(iii): El orden en que el algoritmo de Kosaraju procesa los nodos corresponde al ordenamiento por tiempo de finalización decreciente. Para el DFS de la parte 1, el orden es

$B, E, A, C, D, F, G, I, J, H.$

(iv): Las componentes y representante de cada componente marcados en el grafo G^T :



Rúbrica:

■ (i) Intervalos DFS. (0.6 pts)

- Los intervalos son coherentes con una ejecución válida de DFS sobre el grafo. (0.6 pts)

- (ii) **Grafo transpuesto. (0.6 pts)**
 - Invierte correctamente las aristas del grafo original. (0.5 pts)
 - Presenta el grafo transpuesto de forma clara, ya sea mediante dibujo o alguna otra representación válida (e.g., lista o matriz de adyacencia). (0.1 pts)
- (iii) **Orden de los nodos usado por Kosaraju. (0.8 pts)**
 - Obtiene el orden consistente a partir de los tiempos del primer DFS. (0.5 pts)
 - Usa dicho orden de forma consistente para procesar el grafo transpuesto. (0.3 pts)
- (iv) **Componentes fuertemente conectadas y representantes. (1.0 pts)**
 - Identifica correctamente las componentes fuertemente conectadas. (0.5 pts)
 - Indica un representante válido para cada componente. (0.5 pts)

II. (3 pts) Una red se modela como un grafo dirigido $G = (V, E)$. Los nodos de la red pueden estar activados o desactivados. Si decidimos activar un nodo v , también se activan todos los nodos u tal que hay un camino desde v a u .

Escriba un algoritmo eficiente tal que, dado un grafo dirigido cualquiera $G = (V, E)$, encuentre un conjunto mínimo de vértices iniciales S cuya activación permita activar todos los vértices del grafo. Muestre la complejidad temporal de su algoritmo.

Solución:

La idea es calcular primero las componentes fuertemente conectadas de G . Esto se debe a que al activar un nodo de una componente fuertemente conectada, eventualmente se activan todos los nodos de dicha componente, ya que todos son alcanzables entre sí.

A continuación se construye, de manera explícita o implícita, el grafo G^{CFC} de componentes fuertemente conectadas de G . Tal como se estudió en clases, en este grafo cada nodo representa una componente fuertemente conectada de G , y existe una arista de una componente C_1 a una componente C_2 si existe en G una arista (u, v) tal que $u \in C_1$, $v \in C_2$ y $C_1 \neq C_2$.

Luego se identifican las componentes fuente de G^{CFC} , es decir, las componentes que no tienen aristas entrantes. Es necesario activar al menos un nodo de cada componente fuente, ya que ninguna de ellas puede ser activada desde otra componente.

Por otro lado, basta con activar un nodo de cada componente fuente. El grafo G^{CFC} es acíclico y toda componente es alcanzable desde alguna componente fuente. Por lo tanto, al activar un representante de cada componente fuente, eventualmente se activan todas las componentes de G^{CFC} y, en consecuencia, todos los nodos de G .

Así, el conjunto mínimo S se obtiene escogiendo un representante arbitrario de cada componente fuente de G^{CFC} .

Sea $\text{rep}(v)$ el representante de la componente fuertemente conectada a la que pertenece v .

```

1 [1] CFC ← KOSARAJU( $G$ )  $R \leftarrow \emptyset$  for cada vértice  $v \in V$  do
2    $R \leftarrow R \cup \{\text{rep}(v)\}$ 
3 for cada representante  $r \in R$  do
4   fuente[ $r$ ] ← TRUE
5 for cada arista  $(u, v) \in E$  do
6    $\text{rep}(u) \neq \text{rep}(v)$  fuente[ $\text{rep}(v)$ ] ← FALSE
7  $S \leftarrow \emptyset$  for cada representante  $r \in R$  do
8   fuente[ $r$ ] = TRUE  $S \leftarrow S \cup \{r\}$ 
9 return  $S$ 

```

El algoritmo retorna un representante de cada componente fuertemente conectada que sea una fuente del grafo G^{CFC} . Activar dichos representantes permite activar todo el grafo y la cantidad de vértices retornados es mínima.

El tiempo de ejecución es $O(|V| + |E|)$, asumiendo que G está representado mediante listas de adyacencia. El cálculo de las componentes fuertemente conectadas toma tiempo lineal, y luego se recorren los vértices y las aristas una vez para identificar las componentes fuente.

Tener en cuenta:

- Se aceptan soluciones basadas en un orden topológico del grafo G^{CFC} (es decir, visitan los nodos del grafo G^{CFC} usando un orden topológico), en lugar de encontrar las componentes fuente.
- Un algoritmo que sólo realiza DFS desde vértices no visitados y retorna las raíces del bosque DFS recibe puntaje parcial (máximo 1.0 pts.) solamente si está bien descrito. No obtiene puntaje completo porque el resultado puede depender del orden de visita y no garantiza minimalidad en grafos dirigidos generales. Por ejemplo, para un grafo que tiene una única arista $A \rightarrow B$, la respuesta debería ser el vértice A . Pero si el DFS comienza por B , entregaría A y B como respuesta.

Rúbrica:

- **Uso de componentes fuertemente conectadas. (0.8 pts)**
 - Calcula o utiliza correctamente las componentes fuertemente conectadas de G , explícita o implícitamente. (0.8 pts)
- **Grafo de componentes C^{CFC} y nodos fuentes en el grafo C^{CFC} . (1.0 pts)**
 - Construye explícita o implícitamente el grafo C^{CFC} de las componentes fuertemente conectadas. (0.4 pts)
 - Identifica correctamente las componentes fuente del grafo condensado, es decir, las componentes con grado de entrada cero. (0.4 pts)
 - Determina que el número mínimo de vértices iniciales corresponde al número de componentes fuente. (0.2 pts)
- **Construcción del conjunto S (nodos a activar). (0.8 pts)**
 - Selecciona un representante válido de cada componente fuente. (0.5 pts)
 - Retorna correctamente el conjunto S formado por dichos representantes. (0.3 pts)
- **Tiempo de ejecución. (0.4 pts)**
 - Indica correctamente que el algoritmo puede implementarse en tiempo $O(|V| + |E|)$ usando listas de adyacencia. (0.4 pts)

2. Tablas de Hash

Ud. es el responsable de la infraestructura de datos de una red que recibe mediciones de miles de estaciones meteorológicas (temperatura, humedad y presión) distribuidas por todo el mundo. Cada estación tiene un código identificador único de 10 caracteres con el formato **RRSSS-NNNNN** (ejemplo: **CLSTG-00123**), donde los primeros cinco caracteres representan la región y subregión, y los últimos cinco son un número secuencial. Para procesar y actualizar rápidamente las métricas de cada estación, ud. ha decidido implementar una **tabla de hash**. Dado al volumen de estaciones y que estas están concentradas en las mismas regiones, muchos de los identificadores comparten prefijos idénticos.

a) (1,5 pts.)Diseño de la Función de Hash.

- Proponga una función de hash $h(ID)$ que se haga cargo de distribuir las estaciones de modo que reduzca la cantidad de colisiones. Hint: cada letra puede representarse como un número en un diccionario (Ejemplo: ASCII)

Solución:

Propuesta de función de hash

Dado que los identificadores tienen la forma:

$$ID = RRSSS-NNNNN$$

y muchas estaciones comparten prefijos idénticos, conviene utilizar una función que considere todos los caracteres y su posición dentro de la cadena.

Sea c_i el código ASCII del carácter i -ésimo ($i=0..9$ del identificador).

Definimos:

$$k = \sum_{i=0}^9 c_i \cdot 31^i$$

y posteriormente:

$$h(ID) = k \bmod m$$

donde m corresponde al tamaño de la tabla.

Esta función presenta las siguientes ventajas:

- Asigna distinto peso según la posición, puede no ser otro primo (7 pero 31 separa mejor los valores)
- al separar los valores identifica adecuadamente identificadores con prefijos iguales.
- Reduce la probabilidad de colisiones respecto de una simple suma de códigos ASCII.

Pueden usar otra función pero debe distribuir bien los valores con iguales prefijos

$$ID = RRSSS-NNNNN$$

- Explique por qué, al utilizar el hash modular ($k \bmod m$), se debe evitar que el tamaño de la tabla m sea una potencia de 2 y qué tipo de número se recomienda usar en su lugar para mejorar la dispersión.

Solución:

$$h(k) = k \bmod m$$

y $m = 2^p$, el cálculo del módulo depende únicamente de los últimos p bits de k .

Esto provoca que:

- Los bits más significativos no influyan en la posición final.
- Claves distintas puedan concentrarse en las mismas posiciones.
- Aumenten las colisiones cuando existen patrones en las llaves.

b) (2,5 pts.) Para optimizar el uso de memoria se ha optado por el direccionamiento abierto.

- Escribe el pseudocódigo para la función `INSERTAR(tabla, ID, vID)`, empleando la estrategia de hashing doble.

Solución:

En rigor además del ID lo que se deben ingresar el ID+valores, definimos vID. (tomar ambos casos como correctos)

$$h_1(k) = k \text{ mód } m$$

$$h_2(k) = 1 + (k \text{ mód } (m - 1))$$

$$h_2(k)$$

puede ser otra.

La secuencia de exploración será:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \text{ mód } m$$

`INSERTAR(T, ID)`

```

1  k <- HASH(ID)
2  i <- 0
3  WHILE i < m {
4      pos <- (h1(k) + i*h2(k)) mod m
5      si T[pos] = NULL o T[pos] = BORRADO entonces
6          T[pos] <- ID
7          RETURN TRUE
8      fin si
9      i <- i + 1
10 }
11 RETURN FALSE
```

- Explica por qué al dar de baja una estación, insertar el valor vacío (NULL) genera problema con las búsquedas posteriores y cómo se puede solucionar.

Solución:

Suponga que dos elementos colisionan:

- *A* queda almacenado en la posición *X*.
- *B* colisiona con *A* en *X* y queda almacenado en la posición *Y*.

Si posteriormente se elimina *A* reemplazándolo por NULL, la búsqueda de *B* se detendrá al llegar a la posición *Y*, indicando erróneamente que *B* no existe.

La solución consiste en utilizar una marca de BORRADO:

Entonces cuando se borre *A* no aparecerá como NULL y la búsqueda puede continuar.

Para las futuras inserciones puede reutilizar la posición *X*

c) (2 pts.) Suponga que el sistema cuenta con una tabla de tamaño $B = 10.000$.

- Escribe el proceso de rehashing que debe ejecutarse cuando el factor de carga supera el 0,8. Indica cuál es la complejidad en tiempo de esta operación y si el tiempo de ejecución depende de la cantidad de elementos (N) o del tamaño total de la tabla (B). **Solución:**
El factor de carga de una tabla hash es:

$$\lambda = \frac{N}{B}$$

donde:

- N : cantidad de elementos almacenados.
- B : tamaño de la tabla.

Cuando:

$$\lambda > 0,8$$

se debe ejecutar el siguiente proceso de *rehashing*:

- Crear una nueva tabla de mayor tamaño.
- Escoger un nuevo tamaño primo.
- Recorrer la tabla antigua.
- Reinsertar cada elemento en la nueva tabla utilizando la nueva función hash.
- Liberar la tabla anterior.

Complejidad Si existen N elementos almacenados, cada uno debe ser reinsertado una vez.

Por lo tanto:

$$T(N) = O(N)$$

- Si la tabla tiene almacenado $N = 9.000$ y se desea reducir el factor de carga a 0,5 mediante un proceso de rehashing, determine el tamaño mínimo que debe tener la nueva tabla. Si posteriormente se insertan 3.000 registros adicionales, indique el nuevo factor de carga y discuta si sería recomendable ejecutar nuevamente un rehashing.

Solución:

Inicialmente:

$$N = 9000$$

Se desea un factor de carga:

$$\lambda = 0,5$$

Entonces:

$$0,5 = \frac{9,000}{B'}$$

Despejando:

$$B' = \frac{9000}{0,5} = 18,000$$

Por lo tanto, el tamaño mínimo requerido es:

$$B' = 18,000$$

En una implementación real se utilizaría un primo cercano, por ejemplo:

$$18013$$

Inserción de 3000 registros adicionales La nueva cantidad de elementos será:

$$N' = 9,000 + 3000 = 12,000$$

El nuevo factor de carga es:

$$\alpha' = \frac{12,000}{18,000}$$

$$\alpha' = 0,6667$$

Por lo tanto:

$$\boxed{\lambda' \approx 0,67}$$

Dado que:

$$0,67 < 0,8$$

todavía no se alcanza el umbral definido para realizar un nuevo rehashing.

Entonces:

No es recomendable ejecutar nuevamente un proceso de rehashing, ya que el factor de carga permanece bajo el límite establecido y el rendimiento esperado de la tabla continúa siendo adecuado.

3. Backtracking

La Federación Unida de Planetas enfrenta una amenaza diplomática en la Zona Neutral Romulana (zona desmilitarizada entre la Federación y el Imperio Romulano). El Comando de la Flota enviará la nave “USS Enterprise” y para ello debe asignar tripulantes en seis puestos críticos del puente: Comando, Ciencia, Ingeniería, Comunicaciones, Navegación, Medicina y un tripulante a cargo de las negociaciones diplomáticas (Embajador) el cual es independiente del puente. Los tripulantes disponibles para estos puestos son: Kirk, Spock, McCoy, Uhura, Scotty, Chekov, Sulu, Picard, Riker, Data, Worf, Geordi y Crusher. Para la asignación de los puestos el Comando de la Flota debe considerar que:

- Si Kirk está en el puente, Picard solo puede ser Embajador.
- Si Picard está en el puente, Kirk no puede estar en el puente.
- Si Sulu está en Comando, Riker no puede estar en el puente.
- Si Riker está en Comando, Kirk no puede estar en el puente.
- Si Kirk está en el puente, Worf no puede estar en el puente.
- Si Uhura está en el puente, Spock debe estar en el puente o ser Embajador.
- Si Crusher está en el puente, McCoy no puede estar en el puente.
- Si Worf está en el puente, Kirk no puede estar en el puente.

Adicionalmente, el Comando de la Flota desea minimizar el “riesgo operacional”, definido como la suma de costos de asignar a cada tripulante a un puesto menos preferido. La matriz C de costos es:

Tripulante	Comando	Ciencia	Ingeniería	Comunicaciones	Navegación	Medicina	Embajador
Kirk	0	50	15	25	5	99	25
Spock	5	0	5	10	5	99	0
McCoy	75	15	75	50	75	0	75
Uhura	15	25	50	0	25	99	5
Scotty	45	15	0	65	25	99	50
Chekov	25	45	15	35	5	99	65
Sulu	0	99	99	45	0	99	45
Picard	0	15	25	10	40	60	5
Riker	5	25	35	20	15	65	15
Data	15	0	10	30	25	30	75
Worf	60	40	40	99	20	99	99
Geordi	50	10	0	40	30	60	99
Crusher	25	25	45	35	15	0	99

En este contexto:

- a) (1 pt.) Modele este problema como un CSP, definiendo variables, dominios y restricciones.
- b) (3 pts.) Aplicando la técnica algorítmica Backtracking, proponga el pseudocódigo de la función set-Crew() para resolver la asignación de puestos que minimiza el riesgo operacional, indicando además los parámetros adecuados para realizar la llamada inicial.
- c) (2 pts.) Explique cómo aplicaría en su solución propagación, de modo de mejorar el desempeño de su algoritmo.

3. Backtracking

[Enunciado]

- a) [1 punto] Modele este problema como un CSP, definiendo variables, dominios y restricciones.

Puntaje	Opción 1	Opción 2
0,3 pts.	Variables: Puestos {Comando, Ciencia... ,Embajador}	Tripulantes { }
0,3 pts.	Dominios: Tripulantes {Kirk, Spock... , Crusher}	Puestos { }
0,4 pts.	Restricciones: - Si Kirk está en el puente, Picard no puede estar en el puente. [+7 restricciones de enunciado] Extra: Cada Tripulante puede ocupar un solo puesto	Equivalentes

- b) [3 puntos] Aplicando la técnica algorítmica Backtracking, proponga el pseudocódigo de la función setCrew() para resolver la asignación de puestos que minimiza el riesgo operacional, indicando además los parámetros adecuados para realizar la llamada inicial.

Puntaje	Pauta
2 pts.	<pre> setCrew(P,T): // {Puestos},{Tripulantes} 1 if P=∅: // Encontramos una solución 2 mP ← minRisk(P,mP) // Comparamos y guardamos la mejor 3 return true 4 p ← un puesto de P // p es el puesto a asignar de P 5 for t ∈ T: // t es un tripulante de T 6 if test(t,p): // si t puede ser asignado a p 7 p ← t // se asigna t a p 8 setCrew(P-{p},T): // intenta asignar el siguiente p 9 p ← ∅ // se intenta siguiente tripulante 10 return mP // la mejor asignación mP </pre>
0,4 pt.	<pre> test(t,p): // vehículo, estacionamiento 1 if t en p no rompe restricciones con los p ya asignados: 2 return true // se puede asignar 3 return false </pre>
0,2 pt.	<pre> minRisk(P,mP): 1 if calcRisk(P) < calcRisk(mP): // mP es la mejor asignación antes 2 mP ← P // P es mejor que el anterior 3 return mP </pre>
0,2 pt.	<pre> calcRisk(P): 1 R ← 0 // matriz de costos C global, R el riesgo 1 for p P: // suma el costo de la asignación P 2 R ← R + C[P,T] // el costo del tripulante asignado al puesto 3 return R </pre>
0,2 pt.	Llamada inicial: setCrew({Puestos},{Tripulantes})

- c) [2 puntos] Explique cómo aplicaría en su solución propagación, de modo de mejorar el desempeño de su algoritmo.

Puntaje	Pauta
1,5 pt.	Al aplicar propagación se busca reducir el dominio de las variables disponibles (no asignadas) dada la asignación válida (cumple las restricciones) realizada, si la asignación falla, se debe des-hacer la propagación. En la función setCrew() esto se realiza a continuación de las líneas 7 y 9 respectivamente.
0,5 pt.	Una opción de propagación es que al asignar válidamente a un tripulante t en el puesto p, se quite el tripulante t del dominio de los puestos pendientes de asignar.

4. Técnicas Algorítmicas

Un club organiza una excursión en canoas por un río y, como es usual, la correcta organización depende de una serie de problemas algorítmicos. En particular, están preocupados por el costo del transporte de las personas por el río. Le piden ayuda a Usted para resolver las siguientes variantes.

1. **(3 pts)** En el primer modelo, se debe asignar a los participantes a las canoas teniendo en cuenta lo siguiente:
 - La excursión cuenta con n personas $p_1, \dots, p_n$, con pesos $w_1, \dots, w_n$.
 - Cada canoa tiene una capacidad máxima de W kilos y puede llevar a lo más dos personas. Se asume que $W \geq w_i$, para todo $i = 1, \dots, n$.
 - El arriendo de cada canoa usada en la excursión tiene un costo fijo de C pesos.
 - Las n personas deben participar de la excursión.

Escriba un algoritmo **codicioso** que permita minimizar el costo total de la excursión. Indique:

- a) la función objetivo que se debe optimizar;
- b) la estrategia codiciosa usada y su justificación;
- c) la complejidad temporal del algoritmo.

Solución:

- a) **Función objetivo:**

Si se usan k canoas, el costo total de la excursión es

$$C \cdot k.$$

Como C es fijo para cada canoa, minimizar el costo total equivale a minimizar la cantidad k de canoas usadas.

- b) **Estrategia codiciosa y justificación:**

La estrategia codiciosa consiste en ordenar las personas por peso y, en cada paso, considerar a la persona más pesada que queda por asignar. Si esta persona puede compartir canoa con la persona más liviana que queda, se asignan ambas a la misma canoa. En caso contrario, la persona más pesada debe ir sola.

[1] Ordenar los pesos de menor a mayor. $i \leftarrow 1$ $j \leftarrow n$ $k \leftarrow 0$ **while** $i \leq j$ **do**

$w_i + w_j \leq W$ Asignar p_i y p_j a una misma canoa. $i \leftarrow i + 1$ $j \leftarrow j - 1$ **else**

Asignar p_j sola a una canoa. $j \leftarrow j - 1$ $k \leftarrow k + 1$ **return** k

La justificación es la siguiente. Consideremos a la persona más pesada que queda. Si no puede compartir canoa con la persona más liviana, entonces no puede compartir canoa con ninguna otra persona, ya que todas las demás pesan al menos lo mismo que la más liviana. Por lo tanto, debe ir sola.

Si la persona más pesada sí puede compartir canoa con la persona más liviana, entonces emparejarlas es una decisión codiciosa válida: la persona más pesada debe ocupar una canoa de todas formas, y usar como acompañante a la persona más liviana deja a las demás personas disponibles para las asignaciones restantes.

- c) **Tiempo de ejecución:**

El tiempo de ejecución es

$$O(n \log n),$$

debido al ordenamiento inicial de los pesos. Luego, el recorrido con dos índices toma tiempo $O(n)$.

Rúbrica:

- **Función objetivo:** 0.4 pts
 - Identifica que el costo total es $C \cdot k$, donde k es el número de canoas usadas y como C es constante, minimizar el costo total equivale a minimizar el número de canoas usadas. (0.4 pts)
- **Estrategia codiciosa:** 1.8 pts
 - Ordena las personas por peso. (0.3 pts)
 - Considera en cada paso a la persona más pesada que queda. (0.9 pts)
 - Si la persona más pesada cabe con la más liviana, las asigna juntas. (0.3 pts)
 - Si no caben juntas, asigna a la persona más pesada sola. (0.3 pts)
- **Justificación de la estrategia:** 0.4 pts
 - Justifica correctamente que si la persona más pesada no cabe con la más liviana, entonces no cabe con ninguna otra persona. (0.2 pts)
 - Justifica que, si ambas caben juntas, emparejar a la más pesada con la más liviana es una decisión codiciosa válida. (0.2 pts)
- **Tiempo de ejecución:** 0.4 pts
 - Indica correctamente tiempo $O(n \log n)$ para ordenar los pesos. (0.4 pts)

II. (3 pts) En el segundo modelo, a lo largo del río hay embarcaderos numerados $0, 1, \dots, m$, en ese orden desde el inicio hasta el final. El recorrido comienza en el embarcadero 0 y debe terminar en el embarcadero m .

Arrendar una canoa desde el embarcadero i hasta un embarcadero posterior j tiene costo $c(i, j)$, con $i < j$. Los arriendos están asignados a concesionarios distintos, por lo que los costos $c(i, j)$ son totalmente arbitrarios. Puede haber tramos largos muy baratos, y otros tramos cortos más caros, y viceversa.

Escriba un algoritmo de **programación dinámica** que permita minimizar el costo total de la excursión. Indique:

- a) Los subproblemas que define.
- b) La ecuación de recurrencia (incluyendo todas sus partes).

No es necesario implementar el almacenamiento de resultados. **Solución:**

a) **Subproblemas:**

Una forma correcta de definir los subproblemas es:

$$OPT(j)$$

como el costo mínimo para llegar desde el embarcadero 0 hasta el embarcadero j .

Otra forma equivalente es definir:

$$OPT(i)$$

como el costo mínimo para llegar desde el embarcadero i hasta el embarcadero final m .

b) **Ecuación de recurrencia:**

Para la primera definición, el caso base es

$$OPT(0) = 0.$$

Para $j > 0$, la última canoa usada para llegar al embarcadero j debe venir desde algún embarcadero anterior i , con $0 \leq i < j$. Por lo tanto,

$$OPT(j) = \min_{0 \leq i < j} \{OPT(i) + c(i, j)\}.$$

En este caso, la respuesta final es

$$OPT(m).$$

Para la segunda definición, el caso base es

$$OPT(m) = 0.$$

Para $0 \leq i < m$, la primera canoa usada desde el embarcadero i debe llevarnos hasta algún embarcadero posterior j , con $i < j \leq m$. Por lo tanto,

$$OPT(i) = \min_{i < j \leq m} \{c(i, j) + OPT(j)\}.$$

En este caso, la respuesta final es

$$OPT(0).$$

Rúbrica:

- **Definición de subproblemas:** **0.8 pts**
 - Define correctamente un subproblema que represente el costo mínimo hasta un embarcadero, o desde un embarcadero hasta el destino. (0.8 pts)
- **Ecuación de recurrencia:** **1.6 pts**
 - Considera correctamente todas las posibilidades para el último tramo del recorrido, o equivalentemente para el primer tramo si define la recurrencia hacia adelante. (0.6 pts)
 - Combina correctamente el costo del tramo elegido con el valor óptimo del subproblema correspondiente. (0.5 pts)
 - Usa un mínimo sobre las alternativas posibles. (0.3 pts)
 - Respeta correctamente los índices, usando solo embarcaderos anteriores o posteriores según corresponda. (0.2 pts)
- **Caso base y respuesta final.** **0.6 pts**
 - Indica correctamente el caso base de la recurrencia. (0.3 pts)
 - Indica correctamente cuál valor entrega la respuesta final. (0.3 pts)